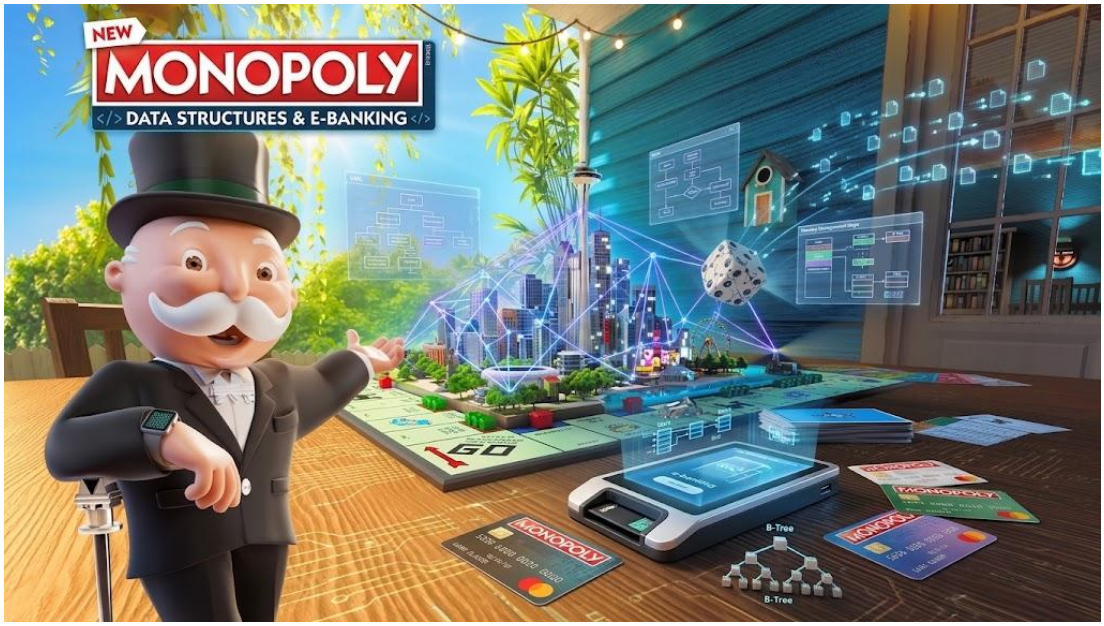
Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Algoritmos y Estructuras de Datos 1
Semestre-II-2026

Proyecto 1
Monopoly Distribuido con Estructuras Lineales



1. Descripción general

El proyecto consiste en desarrollar una versión simplificada de **Monopoly Electrónico** utilizando una arquitectura cliente-servidor, programación orientada a objetos, estructuras de datos lineales y un módulo lector de RFID.
El proyecto será desarrollado en **grupos de 4 estudiantes** y deberá permitir una partida de 4 jugadores.
Durante la demostración deberán utilizarse al menos **2 computadoras físicas diferentes** conectadas mediante una red.
La temática visual del Monopoly será libre (Ejem: TEC, Costa Rica, Videojuegos, Tecnología, Música, Deportes, Películas)

2. Objetivos

El objetivo principal no es recrear todas las reglas del Monopoly tradicional, sino utilizar el juego como contexto para aplicar correctamente estructuras de datos lineales.
El estudiante deberá ser capaz de:

- Aplicar programación orientada a objetos
- Implementar estructuras de datos lineales
- Utilizar estructuras lineales dentro de una aplicación real
- Desarrollar comunicación cliente-servidor
- Administrar múltiples jugadores
- Mantener el estado centralizado del juego
- Registrar y consultar transacciones
- Integrar un dispositivo físico con el sistema.

3. Arquitectura cliente-servidor

El servidor o banco será responsable de mantener el estado oficial de la partida. El jugador organizador de la partida almacena al banco. El banco trabajará de manera automática. Los jugadores, incluido el organizador, deberán solicitar al servidor las acciones que desean realizar. El cliente no podrá modificar directamente:

- saldo
- posición
- propiedades
- turno
- dados
- transacciones.

El banco deberá validar las acciones antes de modificar el estado del juego.

4. Jugadores

Cada jugador deberá tener como mínimo:

- identificador
- nombre
- saldo
- posición actual
- estado activo/inactivo
- propiedades adquiridas.

Las propiedades de cada jugador deberán almacenarse utilizando una estructura lineal implementada por los estudiantes.

5. Tablero

El tablero deberá implementarse mediante una **lista circular doblemente enlazada**.
Cada nodo representará una casilla y deberá contener referencias a:

- casilla anterior
- casilla siguiente.

El tablero deberá contener como mínimo **24 casillas**. Todos jugadores podrán observar el tablero y los movimientos del juego.

6. Tipos de casillas

Deberá existir una clase base Casilla.
Como mínimo deberán implementarse:

- Propiedad
- CasillaEvento
- CasillaEspecial

Cada tipo de casilla deberá implementar su propio comportamiento utilizando herencia y polimorfismo.

7. Propiedades

Cada propiedad deberá contener como mínimo:

- identificador
- nombre
- precio de compra
- alquiler
- propietario

Cuando un jugador cae en una propiedad:

- sí está disponible, podrá comprarla
- sí pertenece a otro jugador, deberá pagar alquiler
- sí pertenece al mismo jugador, no se realiza ningún pago

8. Turnos

Los turnos deberán administrarse utilizando una **cola circular**.
El sistema deberá determinar cuál jugador posee el turno actual y avanzar automáticamente al siguiente jugador cuando termina el turno.
Un jugador no podrá realizar acciones fuera de su turno.

9. Dados

En cada turno el jugador deberá lanzar dos dados electrónicos, especificado más adelante. El resultado deberá transmitirse a los jugadores. El jugador deberá desplazarse a través de los nodos de la lista circular que representa el tablero y mostrarlo a los demás jugadores.

10. Cartas de eventos

Al utilizar una carta, esta deberá pasar al final para poder reutilizarse posteriormente.
Ejemplos de eventos:

- recibir dinero
- pagar dinero
- avanzar posiciones
- retroceder posiciones
- perder un turno
- ir a una casilla determinada.

11. Transacciones

Toda operación económica deberá generar una transacción.
Cada transacción deberá contener como mínimo:

- identificador
- fecha y hora
- número de turno
- tipo
- jugador origen
- jugador destino
- monto
- descripción.

El banco podrá utilizarse como origen o destino.
Tipos mínimos:

- compra de propiedad
- pago de alquiler
- pago al banco
- pago entre jugadores
- ganancia por evento
- pérdida por evento
- premio por pasar por inicio.

12. Historial de transacciones

Las transacciones deberán almacenarse. Se debe seleccionar la estructura adecuada.
La estructura deberá permitir:

- agregar una transacción
- recorrer desde la más antigua
- recorrer desde la más reciente
- buscar por jugador
- buscar por tipo
- imprimir todas las transacciones.

13. Impresión de transacciones

El sistema deberá permitir exportar el historial de la partida.
Formato mínimo obligatorio: TXT
El reporte deberá mostrar como mínimo:

- número de transacción
- turno
- tipo
- origen
- destino
- monto
- descripción.

14. Módulo electrónico (Cajero electrónico)

El proyecto deberá incorporar un módulo electrónico (Se permite entrega en protoboard)
El módulo funcionará como dado electrónico (2 digitos) y lector de tarjetas RFID.
Cada jugador podrá identificarse mediante una tarjeta RFID. Las cuales simulan una billetera electrónica. La tarjeta únicamente deberá identificar al jugador. El saldo oficial deberá mantenerse siempre en el servidor.
Ejemplo de funcionamiento:

1. El juego determina que un jugador debe pagar.
2. El jugador acerca su tarjeta RFID.
3. El dispositivo envía el identificador al servidor.
4. El servidor identifica al jugador.
5. El servidor valida la operación.
6. Se modifica el saldo.
7. Se genera una transacción.
8. Se muestra en pantalla

La billetera física será considerada una extensión del proyecto y no deberá sustituir las estructuras de datos requeridas.
Se recomienda utilizar:

- Arduino Uno – Raspberry Pico 2W
- lector RFID RC522 / Compatible
- Como mínimo 2 tarjetas o llaveros RFID
- 2 Display 7 segmentos
- Botón pulsador
- Resistencias

Referencias:

1. https://naylampmechatronics.com/blog/22_tutorial-modulo-lector-rfid-rc522.html
2. https://programarfacil.com/blog/arduino-blog/display-7-segmentos-dado/
3. (raspberry pi) https://www.instructables.com/How-to-Interface-RFID-RC522-With-Raspberry-Pi/

15. Clases mínimas

El proyecto deberá incluir como mínimo:

- Juego
- Servidor
- Cliente
- Jugador
- Banco
- Tablero
- Casilla
- Propiedad
- CasillaEvento
- CasillaEspecial
- CartaEvento
- Dado
- Transaccion

Además de las clases necesarias para implementar las estructuras.
No se permite utilizar directamente List, LinkedList, Stack, Queue, PriorityQueue o equivalentes para sustituir las estructuras evaluadas.

16. Comunicación cliente-servidor

La comunicación deberá realizarse mediante **sockets TCP**. Los estudiantes deberán definir un protocolo de comunicación.
Ejemplos de acciones:

- CONECTAR
- TIRAR_DADOS
- COMPRAR_PROPIEDAD
- NO_COMPRAR
- TERMINAR_TURNO
- CONSULTAR_ESTADO
- CONSULTAR_TRANSACCIONES

El servidor deberá actualizar a los clientes después de cada acción importante.

17. Validaciones mínimas

El servidor deberá impedir:

- jugar fuera de turno
- comprar sin saldo suficiente
- comprar una propiedad que ya tiene propietario
- lanzar dados múltiples veces en el mismo turno
- realizar pagos con saldo insuficiente sin aplicar la regla de eliminación correspondiente
- modificar directamente información desde el cliente.

18. Fin de la partida

Un jugador podrá quedar eliminado si no puede cubrir un pago obligatorio.
La partida finalizará cuando:

- quede un único jugador activo o
- se alcance una cantidad máxima configurable de turnos.

En caso de alcanzar el límite de turnos, ganará el jugador con mayor patrimonio.
El patrimonio podrá calcularse como:
saldo + valor de propiedades.

19. Entregables

Cada grupo deberá entregar:

1. Código fuente (Enlace al repositorio GitHub).
2. Manual breve de ejecución.
3. Documento técnico formal:
    1. Documentación de las estructuras utilizadas.
    2. Descripción del protocolo cliente-servidor.
    3. Código y conexión del hardware.
    4. Diagrama UML.

20. Defensa del proyecto

Durante la evaluación deberán demostrar como mínimo:

- 4 jugadores conectados
- Comunicación entre al menos 2 computadoras
- Administración de turnos
- Lanzamiento de dados
- Movimiento en el tablero
- Compra de propiedad
- Pago de alquiler
- Ejecución de una carta
- Creación de transacciones
- Uso de estructuras de datos
- Consulta del historial
- Generación del archivo de transacciones

El profesor podrá solicitar modificaciones u operaciones específicas sobre las estructuras durante la demostración.

Rúbrica de evaluación

Nombre de los integrantes|

1 -

2 -

3 -

4 -

Criterio	Porcentaje Calificación
Programación orientada a objetos	10%
Implementación de estructuras de datos y uso correcto dentro del juego.	20%
Funcionamiento cliente-servidor	10%
Lógica y funcionamiento del juego	15%
Módulo de cajero electrónico	10%
El repositorio presenta al menos 10 commits significativos, frecuentes y distribuidos entre los integrantes del grupo. Además	10%
Documentación, diagramas UML y calidad del código	10%
Defensa y demostración	15%
Total	**100%**